Part 1 — Python for AI — The One Question, Explained

The one question: write ONE program that uses all five pillars of Part 1 of the Ultimate AI Book — variables, lists, functions, classes, and files. Questions 1 to 300. Below, each important piece of the code is explained in plain words, with the code itself embedded exactly as written.

1. The variable — one remembered value

The program starts by remembering one value and giving it a name. The name is passing_mark and the value is 60. Every later check in the program uses this one name, so the pass line lives in exactly one place. Change the 60 and the whole program follows the new line.

```
passing_mark = 60                # remember the score needed to pass, and call it passing_mark
```

2. The function — a reusable machine

Next the program builds a small machine named average. You hand it a list of scores, it adds them all together, divides by how many there are, and hands the result back. It is built once here and used later on the real scores, and it would work on any list of numbers you gave it.

```
def average(scores):         # build a reusable machine named average; it receives a list of scores
    return sum(scores) / len(scores)   # add all scores, divide by how many there are, hand back the result
```

3. The class — a blueprint for students

A class is a blueprint, like a mold. This one is named Student. When a student is created, the init part runs and stores that student's name and score on the student itself. The blueprint also carries one built-in action named result. It compares the student's score with passing_mark and hands back the word Pass or the word Fail.

```
class Student:               # a blueprint for making student objects, like a mold
    def __init__(self, name, score):   # runs when a new student is created; stores their name and score
        self.name = name     # keep this student's name on the student itself
        self.score = score   # keep this student's score on the student itself
    def result(self):        # a built-in action every student can do: report pass or fail
        if self.score >= passing_mark:   # if the score reaches the passing mark...
            return "Pass"    # ...the answer is Pass
        return "Fail"        # otherwise the answer is Fail
```

4. The list — and the function at work

Now the program holds four scores in one list, in order: 72, 45, 91 and 60. Then it runs the average machine on that list and prints the result. The sum is 268, divided by 4 scores, so the average is 67.0.

```
scores = [72, 45, 91, 60]    # one list holding four scores in order
print("Average:", average(scores))   # run the average machine on the list and show the result: 67.0
```

5. Making real students from the blueprint

A blueprint alone does nothing until it is used. This line uses the Student mold twice and makes two real students: Abel with a score of 72 and Alicia with a score of 91. Both are kept together in a list named students.

```
students = [Student("Abel", 72), Student("Alicia", 91)]           # use the blueprint twice to make two real students
```

6. Writing the results into a file

The program opens a file named results.txt for writing. The word with is the safety guard: it closes the file properly when the work is done, even if something goes wrong. Inside, the loop walks through the students one by one and writes one line for each: the name, the Pass or Fail answer from the result action, and a new line mark so the next student starts on a fresh line.

```
with open("results.txt", "w") as f:   # open a file for writing; "with" closes it safely when done
    for s in students:       # go through the students one by one
        f.write(s.name + ": " + s.result() + "\n")   # write name, then the pass/fail answer, then a new line
```

7. Reading the file back

Last, the program opens the same file again, this time for reading, and prints everything inside it. That is the proof the file step worked: what was written to disk comes back onto the screen. Abel scored 72 and Alicia scored 91, and both are at or above the passing mark of 60, so both lines say Pass.

```
with open("results.txt", "r") as f:   # open the same file again, this time for reading
    print(f.read())          # read everything in the file and show it on screen
```

The full output

When the whole program runs from top to bottom, this is exactly what it produces.

```
Average: 67.0
Abel: Pass
Alicia: Pass
```